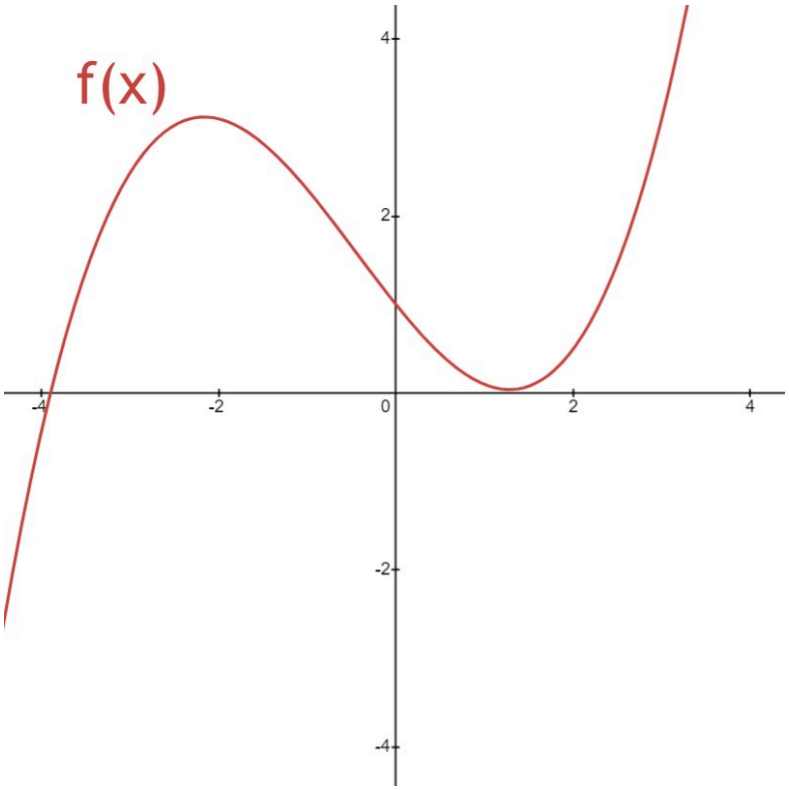
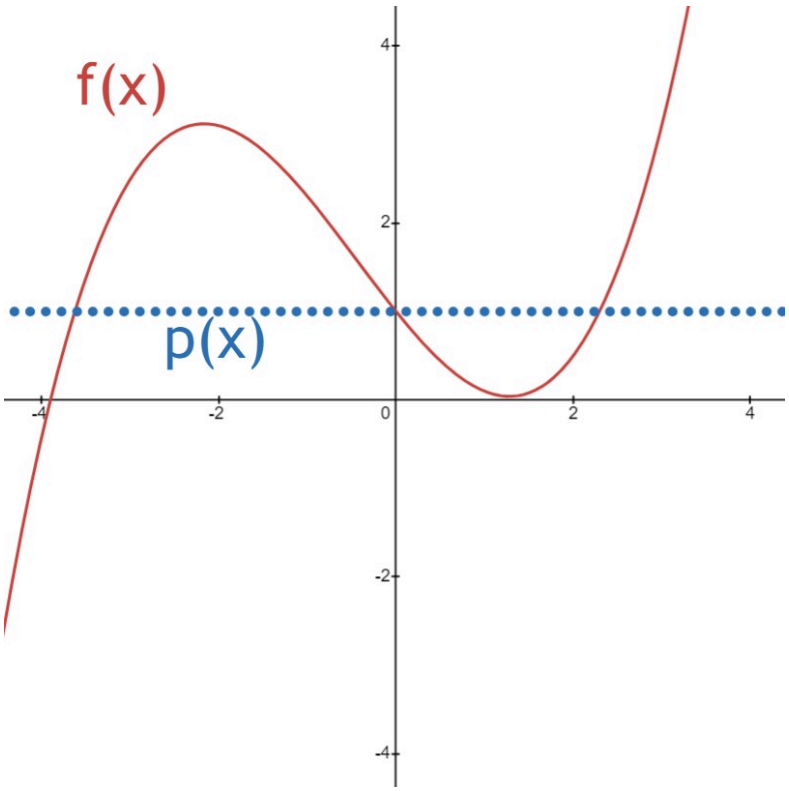
Section 1: Taylor & Maclauren Polynomial Review

Say we have an unknown function $f(x)$ that we want to approximate at $x = 0$ with the function $p(x)$.



The simplist way to do this is to set $p(x) = f(0)$. In this example we can visually see that $f(0) = 1$ thus $p(x) = 1$

Plotted out it would look like this



The approximation $p(x)$ is really acurate when $x = 0$ but not so accurate at other points. One way we could try to improve the approximation is to make the derivative of $p(x)$ equal to the derivative of $f(x)$ at $x = 0$.

Written out formally we want: $p'(0) = f'(0)$

We can visually try to estimate the derivative of f but for this example it will be given as $f'(0) = -1.25$

We can set $p'(x)$ by adding a term to our p function:

$p(x) = f(0) + f'(0) \cdot x \quad \rightarrow \quad p(x) = 1 - 1.25x$

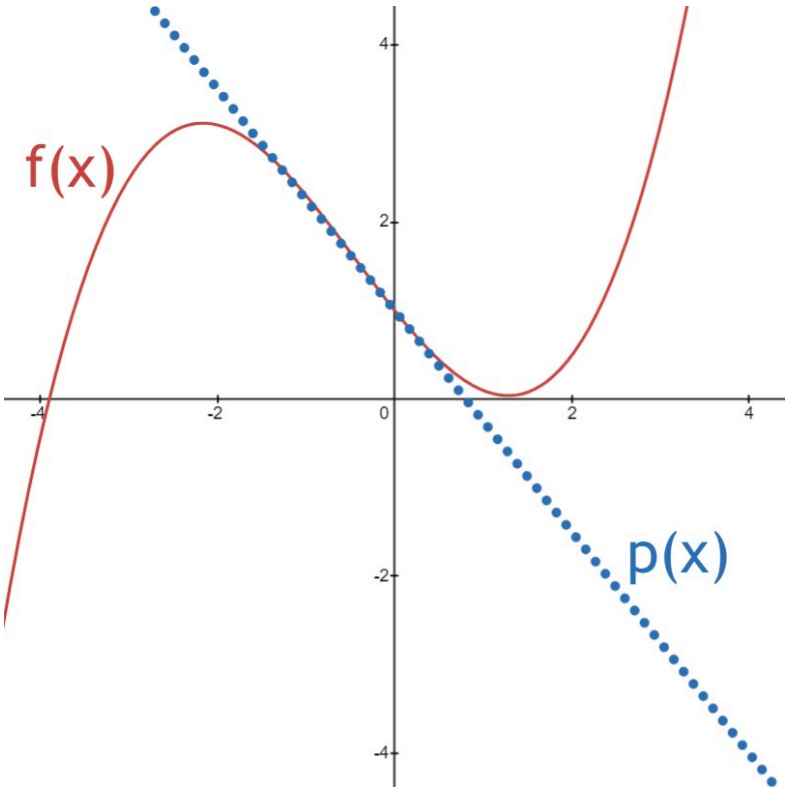
Adding this term doesn't change how $p(x)$ evaluates at $x = 0$ because the new term is multiplied by x.

i.e. solving $p(0) = f(0) + f'(0) \cdot 0 = f(0) \quad \text{or} \quad p(0) = 1 - 1.25 \cdot 0 = 1$

We can also see that the derivative of our new $p(x)$ solves so that we are only left with $f'(0)$:

$p'(0) = 0 + f'(0) \cdot 1 = f'(0)$

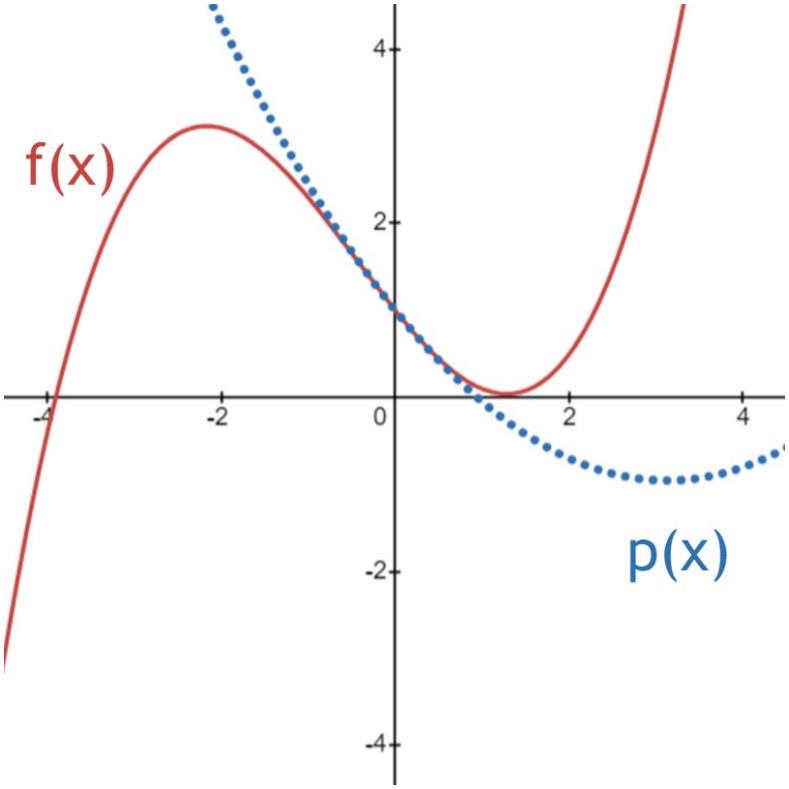
Plotted out it looks like this:



Repeating this process we can try and set the second derivative to be the same: $p''(0) = f''(0)$ with:

$$p(x) = f(0) + f'(0) \cdot x + f''(0) \cdot \frac{1}{2} \cdot x^2$$

While not explicitly computed here, the results of doing this result in the following graph for $p(x)$:



In general, differentiable function $f(x)$ can be approximated at $x = 0$ using n terms as follows:

$$p(x) = f(0) + f'(0)x + \frac{f''(0)}{2!}x^2 + \frac{f'''(0)}{3!}x^3 + \dots \frac{f^{(n)}(0)}{n!}x^n$$

This can be written in summation notation as:

$$p(x) = \sum_{n=0} \frac{f^{(n)}(0)}{n!} \cdot x^n$$

This is the definition of the Maclaurin polynomial.

Exercise 1: Write a Matlab function that can approximate a symbolic function with n terms at the point $x = 0$

```
clear; % Clear old variables (helps with stability)

% A framework for your function is given
function approximate = maclaurin(fun, n)
    % INPUT:
    %      fun: a symbolic function to approximate
    %      n: the order to approximate the function to (starting at 0)
    % OUTPUT
    %      approximate: a symbolic approximation of fun using n+1 terms
    syms x

    %%%%%%%%%%%%%
    %YOUR CODE HERE%
```

```
    %%%%%%%%%%%%%

    %ANSWER KEY (DELETE)%
    approximate = 0;
    for i = 0:n
        approximate = approximate + subs(diff(fun,i),0)/factorial(i) * x^i;
    end
    %%%%%%%%%%%%%%%%

end

% Here is a test case for your function (DO NOT CHANGE)
syms x
f = 5*x^3 - 6*x^2 + 3*x - 4;

% You can experiment with different n values here
n = 1; % Values greater than 3 should not return new answers
maclaurin(f,n)

% Run Tests on your function (DO NOT CHANGE)
assert(maclaurin(f,0) == -4, ' should return -4 i.e. f(0)')
assert(maclaurin(f,1) == 3*x-4, 'This should return 3x-4')
assert(maclaurin(f,2) == -6*x^2+3*x-4,'This should return -6x^2 + 3x - 4')
assert(maclaurin(f,3) == 5*x^3-6*x^2+3*x-4, 'This should return 5x^3 - 6x^2 + 3x -
4')
disp('All Tests Passed!');
```

This would be much more useful if the function could be estimated at a point other then $x = 0$. Fortunately, it can be with a simple modification to the formula! The formula for finding a polynomial estimation of a function at a point c is as follows:

$$p(x) = f(c) + f'(c)(x-c) + \frac{f''(c)}{2!}(x-c)^2 + \frac{f'''(c)}{3!}(x-c)^3 + \ldots \frac{f^{(n)}(c)}{n!}(x-c)^n$$

or

$$p(x) = \sum_{n=0} \frac{f^{(n)}(c)}{n!} \cdot (x-c)^n$$

Exercise 2: Write a Matlab function that can approximate a symbolic function with n terms at an arbitrary point c

```
clear; % Clear old variables (helps with stability)

% A framework for your function is given
function approximate = taylor(fun, n, c)
    % INPUT:
    %      fun: a symbolic function to approximate
```

```
    %      n: the order to approximate the function to (starting at 0)
    %      c: the point at which to approximate fun
    % OUTPUT
    %      approximate: a symbolic approximation of fun at c using n+1 terms
    syms x

    %%%%%%%%%%%%%
    %YOUR CODE HERE%
    %%%%%%%%%%%%%

    %ANSWER KEY (DELETE)%
    approximate = 0;
    for i = 0:n
        approximate = approximate + subs(diff(fun,i),c)/factorial(i) * (x-c)^i;
    end
    %%%%%%%%%%%%%%%%

end

% Here is a test case for your function (DO NOT CHANGE)
syms x
f = 0.1*x^4 + 0.5*x^3 - 3*x - 2;

% You can experiment with different n and c values here
n = 0;
c = 2;
taylor(f,n,c)

% Run Tests on your function (DO NOT CHANGE)
assert(taylor(f,0,0) == -2, ' should return 2')
assert(taylor(f,0,2) == -12/5, 'This should return -12/5 or -2.4')
assert(simplify(taylor(f,1,1)) == -11/10*x-33/10, 'This should return -11x/10 -
33/10')
assert(simplify(taylor(f,2,-2)) == (-3/5)*x^2-(13/5)*x-(6/5),'This should return
(-3/5)*x^2 - (13/5)*x - (6/5)')
disp('All Tests Passed!');
```